Experiment 7

Title: Implementation of Binary Search Tree Operations

Objectives:

To understand the structure and properties of a Binary Search Tree (BST). To implement fundamental operations on a BST: insertion, deletion, display (in-order, pre-order, post-order traversal), and search. To analyze the time and space complexity of these operations.

Problem Statement:

Develop a program that effectively manages a collection of data using a Binary Search Tree, allowing users to perform various operations such as adding new elements, removing existing elements, searching for specific elements, and viewing the tree's contents in different traversal orders.

Outcomes:

A functional program capable of performing all specified BST operations. A clear understanding of BST data structure and its applications. Proficiency in implementing recursive algorithms for tree traversals.

Software and Hardware Requirements:

Software: C++ compiler (e.g., g++), Integrated Development Environment (IDE) like VS Code or Code::Blocks. Hardware: A computer system with a minimum of 4GB RAM and a modern processor.

Date of Completion: October 6, 2025

Assessment Grade/Marks and Assessor's Sign: (To be filled by assessor)

Theory Concept in Brief:

A Binary Search Tree (BST) is a node-based binary tree data structure which has the following properties:

The left subtree of a node contains only nodes with keys lesser than the node's key.

The right subtree of a node contains only nodes with keys greater than the node's key.

Both the left and right subtrees must also be binary search trees.

There must be no duplicate nodes.

Algorithm:

Insertion:

If the tree is empty, create a new node and make it the root.

If the new node's value is less than the current node's value, recursively insert into the left subtree.

If the new node's value is greater than the current node's value, recursively insert into the right subtree.

Deletion:

Find the node to be deleted.

Case 1: Node has no children: Simply remove the node.

Case 2: Node has one child: Replace the node with its child.

Case 3: Node has two children: Find the in-order successor (smallest in right subtree) or predecessor (largest in left subtree), copy its value to the node to be deleted, and then delete the successor/predecessor.

Display (In-order Traversal): Traverse the left subtree, Visit the root, and Traverse the right subtree.

Search:

If the tree is empty or the current node's value matches the search value, return the current node.

If the search value is less than the current node's value, recursively search the left subtree.

If the search value is greater than the current node's value, recursively search the right subtree.

Flowchart: (A visual representation of the algorithms for insertion, deletion, display, and search would be presented here, using standard flowchart symbols for processes, decisions, input/output, and start/end points.)

Test Cases:

Insertion: Insert elements.

Search: Search for existing element (40), non-existing element (90).

Deletion: Delete a leaf node (20), a node with one child (60), a node with two children (50).

Display: Perform in-order, pre-order, and post-order traversals on the tree after insertions and deletions.

Test Data Set:

Initial tree: empty Insertion sequence: 50, 30, 70, 20, 40, 60, 80 Search values: 40, 90 Deletion values: 20, 60, 50

Mathematical Model (if applicable):

The height of a BST can impact the time complexity of operations. In the worst case (skewed tree), the height can be O(N), leading to O(N) complexity for operations. In the best case (balanced tree), the height is O(log N), resulting in O(log N) complexity.

Conclusion/Analysis of Implementation:

The implementation successfully demonstrates the core operations of a Binary Search Tree. The choice of recursive algorithms for insertion, deletion, and traversals provides a concise and elegant solution. The time complexity of operations is dependent on the tree's balance, ranging from O(log N) for a balanced tree to O(N) for a skewed tree. Further enhancements could include self-balancing mechanisms like AVL trees or Red-Black trees to guarantee O(log N) performance for all operations. The test cases confirm the correctness of each implemented operation.